Proyecto de Programación en Python

Desafío de desarrollo

Durante las próximas dos semanas desarrollarás un proyecto propio en Python, para poner en práctica lo aprendido e **investigar nuevas herramientas por cuenta propia**.

La idea no es solo hacer un programa que funcione. El objetivo es que seas capaz de:

- Identificar un problema o necesidad.
- Pensar una solución y dividir el problema en partes.
- Programar la solución, probarla y corregir errores.
- Investigar cuando no sepas cómo hacer algo.
- Explicar las decisiones que tomaste durante el desarrollo.

Puedes trabajar **individualmente o en un grupo pequeño**.

¿Qué puedes crear?

El tema es libre. Puedes partir de un problema real, algo que te interese o una idea que quieras convertir en programa. Por ejemplo:

- Un juego o una simulación.
- Una herramienta para organizar o administrar información.
- Un sistema para resolver un problema cotidiano.
- Una aplicación de deportes, música, videojuegos o películas.
- Una herramienta educativa o una aplicación gráfica.
- Cualquier otra idea que puedas desarrollar en Python.

Conversa con otras personas para descubrir problemas que un programa pueda solucionar. Por ejemplo:

> *«¿Hay alguna tarea que haces frecuentemente y que sería más fácil si existiera un programa que la realizara?»*

Esa respuesta puede convertirse en tu proyecto.

El desafío

Tu proyecto debe ser **más que un programa de una sola función**. Debe tener **varios procesos que trabajen juntos**.

No buscamos proyectos como:

- Piedra, papel o tijera.
- Un conversor simple o una calculadora básica.
- Tres en línea sin funcionalidades adicionales.
- Un programa que solo hace preguntas y entrega una respuesta.

Puedes usar cualquiera de estas ideas como punto de partida, pero deberás llevarla a un nivel mayor de complejidad.

Tipo de proyecto	Funciones que podría combinar
Sistema de inventario	Agregar, eliminar, buscar y modificar productos; consultar cantidades; calcular valores; mostrar estadísticas
Juego	Niveles, puntuación, vidas, enemigos, objetos, dificultad progresiva, condiciones de victoria y derrota

No necesitas implementar exactamente estas funciones: son ejemplos del nivel de complejidad que buscamos.

Algunas ideas para comenzar

- **Sistema de biblioteca:** registrar, buscar, prestar y devolver libros; consultar disponibilidad.
- **Sistema de inventario:** agregar, eliminar, buscar y modificar productos; controlar cantidades; calcular valores; mostrar productos con poco stock.
- **Sistema de estudiantes:** registrar estudiantes y notas; calcular promedios; mostrar aprobados y reprobados; estadísticas del curso.
- **Juego de aventura:** elegir caminos, conseguir objetos, enfrentar enemigos, tener estadísticas y diferentes finales según las decisiones.
- **Juego con Pygame:** plataformas, combate, carreras, supervivencia, puzzle, aventura o estrategia. Agrega mecánicas propias y no te limites a modificar un ejemplo existente.
- **Aplicación con Tkinter:** gestor de tareas, agenda, sistema de notas, inventario, calculadora avanzada, registro de información o aplicación educativa.

También puedes crear tu propia idea. Lo importante es que puedas responder:

1. ¿Qué problema estoy intentando resolver?
2. ¿Qué debe hacer mi programa para resolverlo?
3. ¿Quién utilizaría mi programa?
4. ¿Qué información necesita manejar?
5. ¿Qué decisiones debe tomar el programa?

Requisitos mínimos

Tu proyecto debe usar Python e incorporar los conceptos trabajados en clases, cuando corresponda:

- Variables, `print()` e `input()`.
- Condicionales: `if`, `elif`, `else`.
- Ciclos `for`.
- Listas.

No los uses de manera artificial: úsalos porque **son necesarios para resolver el problema que elegiste**.

Puedes aprender nuevas herramientas

No tienes que limitarte a lo visto en clases. Si tu proyecto lo necesita, investiga, por ejemplo:

- Funciones, diccionarios, módulos y clases.
- Manejo de archivos: *«Quiero guardar los datos cuando cierre el programa»*.
- Tkinter: *«Quiero crear una interfaz gráfica»*.
- Pygame: *«Quiero agregar gráficos o movimiento»*.
- Otras librerías o herramientas que investigues.

La investigación y el aprendizaje autónomo **son parte del proyecto**.

Una condición importante

Si usas una herramienta o concepto que no hemos trabajado en clases, debes **entender cómo funciona**. No tienes que memorizar la documentación, pero sí explicar:

- Qué hace.
- Para qué lo utilizaste.
- Cómo funciona dentro de tu proyecto.
- Qué problema te ayudó a resolver.

Uso de Inteligencia Artificial

Puedes usar IA como herramienta para aprender y resolver problemas. Por ejemplo, para:

- Entender un error o encontrar errores en tu código.
- Que te explique una parte del código.
- Aprender una librería o investigar conceptos nuevos.
- Buscar alternativas para resolver un problema.
- Revisar tu código o pedir ejemplos.

> ***La IA no debe construir el proyecto completo por ti.***

Si usas código generado o modificado con ayuda de IA, debes entenderlo. Durante la presentación el profesor puede preguntarte:

- «¿Qué hace esta parte del código?»
- «¿Por qué decidiste hacerlo de esta manera?»

No basta con que el programa funcione: debes poder **explicar el proceso que usaste para construirlo**.

Piensa en procesos, no solo en código

Antes de programar, piensa en qué tiene que ocurrir dentro de tu programa. Por ejemplo, en una tienda:

Usuario → selecciona producto → verifica stock → calcula precio → confirma compra → actualiza inventario → muestra resultado

Ese proceso puede transformarse después en código. No necesitas entregar un diagrama de flujo formal, pero sí **entender y explicar cómo funciona el proceso de tu programa**.

Desarrollo del proyecto

El proyecto dura aproximadamente **dos semanas**, en cuatro etapas:

1. **Primera clase (4 horas):** definir el proyecto y conseguir una primera versión funcional.
2. **Durante la semana:** continuar el desarrollo fuera del horario de clases.
3. **Segunda clase (4 horas):** terminar, probar y preparar la presentación.
4. **Clase final:** presentación de los proyectos.

Aprovecha las horas de clase para avanzar, probar tu programa, investigar y resolver dudas.

Primera clase: comenzar

1. Definir tu proyecto y formar el grupo, si corresponde.
2. Definir el problema o propósito.
3. Identificar las principales funcionalidades.
4. Pensar cómo funcionará cada proceso.
5. Comenzar a programar y conseguir una primera versión funcional.

No intentes construir todo de una vez. Primero consigue una versión básica; después agrega mejoras.

Durante la semana

Continúa desarrollando tu proyecto. Puedes agregar funcionalidades, corregir errores, mejorar el diseño o la experiencia del usuario, investigar nuevas herramientas, agregar una interfaz, sonidos o gráficos, y probar diferentes soluciones.

Segunda clase: terminar y preparar

- Terminar las funcionalidades principales.
- Corregir errores, realizar pruebas y agregar mejoras.
- Preparar la demostración y la presentación.

Al finalizar esta etapa debes tener una versión funcional que puedas presentar.

Presentación final

Cada estudiante o grupo tendrá aproximadamente **5 minutos**. La presentación debe ser breve y concreta. Debes explicar:

1. **¿Qué hicieron?** Presenta brevemente el proyecto.
2. **¿Qué problema resuelve?** Cuál era el objetivo del programa.
3. **¿Cómo funciona?** Los principales procesos de tu programa.
4. **¿Qué utilizaron?** Por ejemplo `if`, `for`, listas, funciones, Tkinter, Pygame, archivos u otras librerías.
5. **¿Qué funcionalidades implementaron?** Las características principales.
6. **Demostración.** Muestra el programa funcionando.
7. **¿Qué fue lo más difícil?** Un problema que tuvieron que resolver.

8. **¿Qué aprendieron?** Puede ser algo de Python, una librería, una técnica de programación o una forma distinta de abordar un problema.

¿Qué se evaluará?

Aspecto	Pregunta guía
Comprensión del problema	¿El grupo entiende qué problema intenta resolver?
Diseño de la solución	¿Fue capaz de dividir el problema en diferentes procesos?
Aplicación de Python	¿Utiliza correctamente los conceptos aprendidos?
Desarrollo	¿El programa tiene varias funcionalidades relacionadas entre sí?
Resolución de problemas	¿Fue capaz de investigar, probar alternativas y corregir errores?
Aprendizaje	¿Fue capaz de incorporar conocimientos o herramientas nuevas?
Comprensión del código	¿Puede explicar las partes principales de su programa?
Presentación	¿Puede explicar claramente qué hizo y demostrar cómo funciona?

Una última recomendación

No intentes hacer el proyecto más grande posible. Es mejor un proyecto con varias funcionalidades que funcione y que puedas explicar, que algo enorme que no alcances a terminar. Empieza con una versión sencilla y agrega funcionalidades después.

Problema → Solución → Procesos → Código → Pruebas → Mejoras

Ficha inicial del proyecto

Antes de comenzar el desarrollo, completen esta ficha:

Campo	Respuesta
Nombre del proyecto	
Integrantes	
¿Qué vamos a crear?	
¿Qué problema queremos resolver?	
¿Quién utilizaría nuestro programa?	
Principales funcionalidades (5)	
¿Qué información deberá manejar nuestro programa?	
Herramientas o tecnologías (además de Python)	
¿Qué nos gustaría investigar o aprender?	
Funcionalidades adicionales si hay tiempo (3)	

El objetivo

No buscamos que escribas la mayor cantidad de código. Buscamos que demuestres que puedes tomar una idea, **convertirla en un problema programable, construir una solución, enfrentarte a los errores y entender lo que desarrollaste**.

El proyecto es tu oportunidad para demostrar lo que puedes hacer con Python.